The AAA Algorithm and Its Extensions: Continuum AAA, p-AAA, and AAA^2

Qiu Jiaxin

March 8, 2026

Abstract

The AAA (Adaptive Antoulas–Anderson) algorithm [1] has emerged as a powerful and versatile tool for rational approximation of functions on real and complex domains. Its numerical stability, adaptive order selection, and absence of initial pole guesses have led to widespread adoption across numerous scientific and engineering disciplines. This paper provides a comprehensive overview of the AAA algorithm and three of its most significant extensions: continuum AAA, p-AAA, and AAA^2. We first review the core AAA algorithm, detailing its barycentric representation, greedy selection strategy, and linearized least-squares formulation. We then examine continuum AAA, which eliminates the need for user-supplied discrete grids by adaptively generating sample points based on the current support point distribution, enabling direct approximation on continuous domains with automatic handling of singularities. Next, we present p-AAA, which extends AAA to multivariate functions, particularly parametric dynamical systems, through a two-variable barycentric form and a block-structured Loewner matrix, allowing independent adaptive order selection in each variable. Finally, we discuss AAA^2, a variant developed for circuit and electromagnetic modeling that enforces real coefficients and stable poles by incorporating conjugate symmetry into the barycentric representation and implementing appropriate pole selection mechanisms. Through detailed algorithmic descriptions and numerical examples, we demonstrate how each extension preserves the core advantages of AAA while addressing specific limitations of the original method. This unified treatment highlights the adaptability of the AAA framework and its continuing evolution to meet the demands of diverse application areas.

Contents

1 Introduction **3**

2 The AAA Algorithm **3**
2.1 Barycentric representation . . . 3
2.2 Greedy selection and least-squares fitting . . . 3
2.3 Greedy support point selection . . . 4
2.4 Properties and advantages . . . 4
2.5 Extensions . . . 4

3 Continuum AAA as an Extension of AAA **4**
3.1 Limitations of discrete AAA . . . 5
3.2 Core idea of continuum AAA . . . 5
3.3 Algorithmic description . . . 5
3.4 Handling of spurious poles . . . 6
3.5 Relation to the original AAA . . . 6
3.6 Advantages of the continuum approach . . . 6

4 The p-AAA as an Extension of AAA **7**
4.1 Recollection of the AAA algorithm . . . 7
4.2 The p-AAA algorithm for two variables . . . 7
4.3 Relation to AAA . . . 8

5 The AAA^2 as an Extension of AAA **9**
5.1 Recollection of the AAA Algorithm . . . 9
5.2 Enforcing Conjugate Symmetry and Real Coefficients . . . 9
5.3 Modified Loewner Matrix and Least-Squares Solution . . . 10
5.4 Pole Extraction and Stability Enforcement . . . 10
5.5 Relation to AAA . . . 11

6 Conclusions **11**

1 Introduction

Your introduction goes here! Simply start writing your document and use the Recompile button to view the updated PDF preview. Examples of commonly used commands and features are listed below, to help you get started.

Once you're familiar with the editor, you can find various project settings in the Overleaf menu, accessed via the button in the very top left of the editor. To view tutorials, user guides, and further documentation, please visit our help library, or head to our plans page to choose your plan.

2 The AAA Algorithm

2.1 Barycentric representation

The starting point of AAA is the representation of a rational function $r(z)$ not in the conventional polynomial quotient form $p(z)/q(z)$, but as a **barycentric quotient** [**?**]:

$$r(z) = \frac{n(z)}{d(z)} = \frac{\sum_{j=1}^{m} \dfrac{w_j f(s_j)}{z - s_j}}{\sum_{j=1}^{m} \dfrac{w_j}{z - s_j}}, \tag{1}$$

where $s_1, \ldots, s_m$ are a set of **support points** (a subset of the sampling set) and $w_1, \ldots, w_m$ are nonzero **barycentric weights**. This representation is numerically stable even when the zeros and poles of r cluster near singularities—a situation that causes catastrophic cancellation in the polynomial representation because the coefficients of p and q vary over many orders of magnitude. The barycentric form can represent any rational function of degree at most $m - 1$ and is the key to the robustness of AAA.

2.2 Greedy selection and least-squares fitting

Given a discrete sample set $Z = \{z_i\}_{i=1}^{N}$ (typically a discretization of a continuum) and corresponding function values $f_i = f(z_i)$, the AAA algorithm builds an approximant iteratively. At each step, it maintains a partition of the data into two disjoint sets: the **support points** $S = \{s_1, \ldots, s_m\}$ (initially chosen greedily) and the remaining **test points** $T = Z \setminus S$.

- **Interpolation:** The approximant is forced to interpolate f at the support points. This is automatically satisfied by the barycentric form (1) for any nonzero weights, because the singularity at $z = s_j$ is removable and $\lim_{z \to s_j} r(z) = f(s_j)$.

- **Linearized least-squares:** For the test points, the error $f(z) - r(z)$ is nonlinear in the weights. AAA linearizes it by considering the residual of the equation $f(z)d(z) - n(z) = 0$. For each test point $z \in T$,

  $$f(z)d(z) - n(z) = \sum_{j=1}^{m} \frac{f(z) - f(s_j)}{z - s_j} w_j.$$

  Collecting these residuals for all test points yields the overdetermined linear system $A\mathbf{w} \approx 0$, where A is the **Loewner matrix** of size $|T| \times m$ with entries

  $$A_{ij} = \frac{f(z_i) - f(s_j)}{z_i - s_j}, \qquad z_i \in T,\ s_j \in S.$$

  The weight vector $\mathbf{w}$ is then chosen as the right singular vector corresponding to the smallest singular value of A, i.e., the minimizer of $\|A\mathbf{w}\|_2$ subject to $\|\mathbf{w}\|_2 = 1$. This solves the linearized least-squares problem and determines the barycentric weights.

2.3 Greedy support point selection

AAA builds the set of support points incrementally. Starting with an empty set (or with a few initial points), at each iteration it computes the current approximant, evaluates the error $|f(z) - r(z)|$ at all test points, and moves the point with the largest error into the support set. The Loewner matrix is then updated (a new column is added), and new weights are computed via a new SVD. This process continues until a prescribed accuracy is reached or a maximum number of iterations is exceeded. The greedy strategy ensures that the approximation focuses on regions where the error is largest, leading to an adaptive, near-optimal distribution of support points.

2.4 Properties and advantages

The AAA algorithm enjoys several important properties:

- **Numerical stability:** The barycentric representation and the use of SVD make the computation robust even for high degrees and in the presence of near-singularities.
- **No initial guess:** Unlike methods such as vector fitting or the Remez algorithm, AAA requires no initial choice of poles or nodes.
- **Adaptive order:** The algorithm automatically determines a suitable degree by monitoring the error; the user only supplies a tolerance.
- **Versatility:** AAA works for real or complex data, on arbitrary point sets, and can be extended to matrix-valued functions or parametric problems.

The algorithm is remarkably fast. For example, approximating $\exp(z)$ on 100 points on the unit circle to 13-digit accuracy takes about 0.002 seconds on a laptop and yields a degree-7 rational function whose maximum deviation on the unit disk is 2.81×10^{-15}. For more challenging functions, such as the Riemann zeta function on a vertical line, AAA produces a high-quality approximation in a fraction of a second.

2.5 Extensions

Since its introduction, AAA has been extended in several directions. The **AAA-Lawson** variant adds an iteratively reweighted least-squares phase to converge to a minimax (best) approximation. The **continuum AAA** algorithm works directly on a continuum domain without requiring a pre-chosen discrete grid, adaptively refining the sampling as the approximation improves. Most recently, the **AAA-LS** method uses the poles of a rational approximant to construct highly accurate solutions to Laplace and related PDEs, with applications in Stokes flow and beyond.

These developments underscore the central role that the original AAA algorithm has come to play in modern numerical analysis. Its simplicity, efficiency, and robustness make it an indispensable tool for rational approximation.

3 Continuum AAA as an Extension of AAA

The AAA (Adaptive Antoulas–Anderson) algorithm has become a standard tool for rational approximation of functions on real or complex domains. Its original formulation, however, operates on a **fixed discrete set** of sample points supplied by the user. While this discrete paradigm is sufficient for many problems, it has inherent limitations when the target function is defined on a continuum, especially in the presence of singularities. To overcome these limitations, Driscoll, Nakatsukasa, and Trefethen recently introduced the **continuum AAA algorithm** [3], which extends AAA to work directly on continuous domains without requiring a pre-chosen grid. This section details how continuum AAA builds and generalizes the original AAA algorithm.

3.1 Limitations of discrete AAA

In the standard AAA algorithm, the user must provide a discrete set $Z \subset R$ or C of N points together with function values $f(Z)$. The algorithm then constructs a rational approximant in barycentric form by greedily selecting support points from Z and solving a linearized least-squares problem involving a Loewner matrix with N rows. As noted in the continuum AAA paper, this approach is often adequate when one merely wishes to approximate on that specific set. However, if the true goal is approximation on a continuum E (e.g., an interval, a circle, or the imaginary axis), the user must discretize E in advance. This raises several concerns:

- **Philosophical dissatisfaction**: "It is unappealing philosophically to require the user to specify a discrete set rather than the domain that is actually of interest."
- **Efficiency loss**: Although the cost of a tall-and-skinny SVD grows only linearly with N, a large N still incurs a constant factor overhead that could be avoided if the algorithm adapted the sample points automatically.
- **Difficulty near singularities**: Exponential clustering of sample points near singularities is essential for high-accuracy rational approximation. With a fixed grid, the user must manually design clustering (e.g., using `logspace` or `tanh` transformations) and experiment to get it right. This becomes even harder when singularities are at unknown locations, as often occurs in model order reduction when poles lie near the imaginary axis.

3.2 Core idea of continuum AAA

The continuum AAA algorithm eliminates the need for a user-supplied grid by **adaptively constructing sample points at each iteration** based on the current set of support points. At every step, given the current support points $S = [s_1, \ldots, s_m]$, a new sample set X is generated by placing a fixed number p of equally spaced points between each consecutive pair of support points. In the default implementation on $[-1, 1]$, $p = \max\{3,\, 16 - m\}$ (so that many points are used in early iterations). Thus the sample points are never fixed; they evolve with the approximation, "zooming in" on regions where the error is large.

The algorithm proceeds greedily, exactly as in classical AAA: after computing the barycentric weights w via an SVD of the Loewner matrix built from X and S, the next support point is chosen as the sample point where the current error $|f(x_i) - r(x_i)|$ is maximal. This new point is added to S, and the process repeats. The crucial difference is that **the sample points are regenerated after each update of** S, ensuring that the sampling always remains fine enough relative to the current support point distribution.

3.3 Algorithmic description

A concise description of the continuum AAA loop (for the interval $[-1, 1]$) is given in the original paper as:

$S = [-1, 1]$
for $m = 2, 3, \ldots$ until convergence
  $X := \mathbf{xS}(S)$ (generate sample points)
  Use SVD to compute barycentric weights for next approximation $r \approx f$
  $S := S \cup \{\text{a sample point } x_i \in X \text{ where } |f(x_i) - r(x_i)| \text{ is maximal}\}$
end

Here $\mathbf{xS}(S)$ is a function that places p equispaced points between each pair of consecutive support points. The Loewner matrix A has entries

$$a_{ij} = \frac{f(x_i) - f(s_j)}{x_i - s_j},$$

and the weight vector w is taken as the right singular vector corresponding to the smallest singular value of A. The barycentric approximant is then

$$r(x) = \sum_{j=1}^{m} \frac{w_j f(s_j)}{x - s_j} \Bigg/ \sum_{j=1}^{m} \frac{w_j}{x - s_j}.$$

The iteration stops when the maximum error on the current sample set falls below a prescribed tolerance (default 10^{-13}) or when a maximal degree is reached. Additionally, if ten consecutive steps produce "bad poles" (poles inside the approximation domain) and at least two digits of accuracy have been achieved, the algorithm terminates and returns the last successful approximation.

3.4 Handling of spurious poles

One of the most valuable features inherited from AAA and extended in continuum AAA is the detection and management of unwanted poles. At each step, the poles of the current approximant are computed via a generalized eigenvalue problem. If any pole lies in the approximation domain (e.g., in $[-1, 1]$ for real approximation, or inside the unit disk when analyticity is required), that step is flagged as "bad". The algorithm remembers the best pole-free approximation encountered so far. If the iteration ends because of repeated bad steps, it reverts to that saved approximation, guaranteeing that the returned rational function has no poles in the domain of interest.

3.5 Relation to the original AAA

Continuum AAA is a genuine extension of the original AAA algorithm. If one were to apply continuum AAA with a fixed set of support points and a fixed set of sample points (i.e., never adding new support points), the method would reduce to solving a single least-squares problem on that fixed grid—exactly the linearized least-squares step of AAA. More importantly, **if the sample points are kept constant instead of being regenerated, the algorithm becomes identical to the standard discrete AAA** working on that fixed grid. Thus continuum AAA can be viewed as an adaptive refinement strategy built on top of the AAA core.

The authors state: "Step (1) [generating sample points from support points] is the feature that distinguishes continuum AAA from its fixed-grid predecessor." This adaptive sampling is what enables the algorithm to work directly on a continuum without manual discretization.

3.6 Advantages of the continuum approach

The continuum AAA algorithm offers several significant advantages over the traditional discrete version:

- **Freedom from manual discretization**: The user no longer needs to choose a sampling grid; the algorithm automatically samples where needed.
- **Adaptive resolution**: Near singularities, the support points (and consequently the sample points) become exponentially clustered, mirroring the behavior of optimal rational approximants. This clustering emerges naturally from the greedy selection and the adaptive sampling, without any user intervention.
- **Automatic handling of unknown singularities**: Because the algorithm refines the grid where the error is large, it can discover and resolve singularities even when their locations are not known in advance.
- **Guaranteed pole-free approximations**: By monitoring poles and reverting to the last good approximation, the algorithm ensures that the returned rational function is analytic on the desired domain—a crucial property in many applications such as control theory and model order reduction.

Numerical experiments in the continuum AAA paper demonstrate that continuum AAA achieves high accuracy (often close to machine precision) on a wide range of problems, from smooth functions like e^x to challenging singular functions like $|x|$ and $\tan(z^4)$, all within milliseconds to seconds of computation time.

In summary, continuum AAA is a natural and powerful extension of the original AAA algorithm. It inherits the barycentric representation, greedy selection, and linearized least-squares core, while adding an adaptive sampling mechanism that makes it truly suitable for continuum approximation problems. This extension retains the simplicity and efficiency of AAA while eliminating the need for user-supplied grids and improving robustness near singularities.

4 The p-AAA as an Extension of AAA

The AAA (Adaptive Antoulas–Anderson) algorithm has emerged as a powerful tool for data-driven rational approximation of single-variable functions, such as transfer functions of linear dynamical systems. Its success stems from a skillful combination of interpolation and least-squares fitting within a barycentric representation, together with a greedy selection of interpolation points. In many applications, however, the underlying function depends on several parameters – for instance, the transfer function $H(s,p)$ of a parametric dynamical system depends on both the frequency s and a physical parameter p. The **p-AAA (parametric AAA)** algorithm [2] extends the AAA framework to such multivariate settings while preserving its essential features. This section details how p-AAA generalises AAA to two variables; extensions to more variables follow analogously.

4.1 Recollection of the AAA algorithm

Given samples $\{H(s_i)\}_{i=1}^N$ of a single-variable function, AAA constructs a rational approximant in the **barycentric form**

$$\tilde{H}(s) = \frac{n(s)}{d(s)} = \frac{\sum_{i=1}^k \frac{\beta_i}{s-\sigma_i}}{\sum_{i=1}^k \frac{\alpha_i}{s-\sigma_i}}, \tag{2.2}$$

where the support points σ_i are a subset of the sampling points. The data are partitioned into two disjoint sets:

$$\{s_1,\dots,s_N\} = \{\sigma_1,\dots,\sigma_k\} \cup \{\hat{\sigma}_1,\dots,\hat{\sigma}_{N-k}\}, \qquad \{H(s_i)\} = \{g_1,\dots,g_k\} \cup \{\hat{g}_1,\dots,\hat{g}_{N-k}\}.$$

Interpolation at the σ_i is enforced by setting $\beta_i = g_i\alpha_i$ (provided $\alpha_i \neq 0$). For the remaining points $\hat{\sigma}_i$, the error is linearised:

$$H(\hat{\sigma}_i) - \tilde{H}(\hat{\sigma}_i) \approx \hat{g}_i d(\hat{\sigma}_i) - n(\hat{\sigma}_i) = \sum_{j=1}^k \frac{(\hat{g}_i - g_j)\alpha_j}{\hat{\sigma}_i - \sigma_j} = \mathbf{e}_i^\top L\,\mathbf{a}, \tag{2.9}$$

where L is the Loewner matrix and $\mathbf{a} = [\alpha_1,\dots,\alpha_k]^\top$. The coefficient vector $\mathbf{a}$ is then obtained from the linear least-squares problem

$$\min_{\|\mathbf{a}\|_2=1} \|L\mathbf{a}\|_2, \tag{2.10}$$

i.e. $\mathbf{a}$ is chosen as the right singular vector associated with the smallest singular value of L. AAA proceeds iteratively: at each step the next interpolation point σ_{k+1} is selected by a greedy search over the current error, the partition is updated, and the new least-squares problem is solved. The process stops when a prescribed tolerance or a maximal order is reached.

4.2 The p-AAA algorithm for two variables

Now consider a function $H(s,p)$ of two variables and suppose we have its values on a rectangular grid:

$$h_{ij} = H(s_i, p_j), \qquad i = 1,\dots,N,\ j = 1,\dots,M. \tag{3.1}$$

p-AAA represents the approximant in the **two-variable barycentric form**

$$\tilde{H}(s,p) = \frac{n(s,p)}{d(s,p)} = \frac{\sum_{i=1}^k \sum_{j=1}^q \frac{\beta_{ij}}{(s-\sigma_i)(p-\pi_j)}}{\sum_{i=1}^k \sum_{j=1}^q \frac{\alpha_{ij}}{(s-\sigma_i)(p-\pi_j)}}, \tag{3.2}$$

where $\{\sigma_i\} \subset \{s_i\}$ and $\{\pi_j\} \subset \{p_j\}$ are the selected support points. The data are partitioned as

$$\{s_i\} = \{\sigma_1, \ldots, \sigma_k\} \cup \{\hat{\sigma}_1, \ldots, \hat{\sigma}_{N-k}\}, \quad \{p_j\} = \{\pi_1, \ldots, \pi_q\} \cup \{\hat{\pi}_1, \ldots, \hat{\pi}_{M-q}\}, \tag{3.3}$$

and the corresponding samples are organised into four blocks:

$$\left[\begin{array}{c|c} \mathbf{D}_{\sigma\pi} & \mathbf{D}_{\sigma\hat{\pi}} \\ \hline \mathbf{D}_{\hat{\sigma}\pi} & \mathbf{D}_{\hat{\sigma}\hat{\pi}} \end{array}\right],$$

with $\mathbf{D}_{\sigma\pi} = [H(\sigma_i, \pi_j)]$ etc.

Interpolation. To interpolate the values at (σ_i, π_j) we set, analogously to the single-variable case,

$$\beta_{ij} = H(\sigma_i, \pi_j)\, \alpha_{ij} \qquad (\alpha_{ij} \neq 0). \tag{3.4}$$

Then $\tilde{H}(\sigma_i, \pi_j) = \beta_{ij}/\alpha_{ij} = H(\sigma_i, \pi_j)$ because the singularity in (3.2) is removable.

Linearised least-squares fit. For the remaining data (the blocks $\mathbf{D}_{\sigma\hat{\pi}}, \mathbf{D}_{\hat{\sigma}\pi}, \mathbf{D}_{\hat{\sigma}\hat{\pi}}$) we again linearise the error. Taking a point $(\hat{\sigma}, \hat{\pi})$ in $\mathbf{D}_{\hat{\sigma}\hat{\pi}}$ as an example,

$$\begin{aligned} & H(\hat{\sigma}, \hat{\pi}) - \tilde{H}(\hat{\sigma}, \hat{\pi}) \\ \approx\ & H(\hat{\sigma}, \hat{\pi}) d(\hat{\sigma}, \hat{\pi}) - n(\hat{\sigma}, \hat{\pi}) \\ =\ & \sum_{i=1}^{k} \sum_{j=1}^{q} \frac{(H(\hat{\sigma}, \hat{\pi}) - H(\sigma_i, \pi_j))\alpha_{ij}}{(\hat{\sigma} - \sigma_i)(\hat{\pi} - \pi_j)} = \mathbf{e}_{\hat{\sigma}\hat{\pi}}^{\top} L_{\hat{\sigma}\hat{\pi}}\, \mathbf{a}, \end{aligned} \tag{3.6}$$

where $\mathbf{a} = [\alpha_{11}, \ldots, \alpha_{1q} \mid \cdots \mid \alpha_{k1}, \ldots, \alpha_{kq}]^{\top} \in C^{kq}$ and $L_{\hat{\sigma}\hat{\pi}}$ is the **two-dimensional Loewner matrix** of size $(N-k)(M-q) \times kq$ with entries

$$(L_{\hat{\sigma}\hat{\pi}})_{(\hat{\imath},\hat{\jmath}),(i,j)} = \frac{H(\hat{\sigma}_{\hat{\imath}}, \hat{\pi}_{\hat{\jmath}}) - H(\sigma_i, \pi_j)}{(\hat{\sigma}_{\hat{\imath}} - \sigma_i)(\hat{\pi}_{\hat{\jmath}} - \pi_j)}, \tag{3.7}$$

for $\hat{\imath} = 1, \ldots, N-k,\ \hat{\jmath} = 1, \ldots, M-q$. For the blocks $\mathbf{D}_{\sigma\hat{\pi}}$ and $\mathbf{D}_{\hat{\sigma}\pi}$ one obtains analogous Loewner matrices $L_{\sigma\hat{\pi}}$ (of size $k(M-q) \times kq$) and $L_{\hat{\sigma}\pi}$ (of size $q(N-k) \times kq$). Stacking them yields the global matrix

$$L_2 = \begin{bmatrix} L_{\sigma\hat{\pi}} \\ L_{\hat{\sigma}\pi} \\ L_{\hat{\sigma}\hat{\pi}} \end{bmatrix} \in C^{(MN-kq)\times kq}. \tag{3.10}$$

The coefficient vector $\mathbf{a}$ is then determined by solving the linear least-squares problem

$$\min_{\|\mathbf{a}\|_2 = 1} \|L_2 \mathbf{a}\|_2, \tag{3.10}$$

exactly as in AAA (compare with (2.10)).

Greedy selection. p-AAA also proceeds iteratively. Let $\tilde{H}(s, p)$ be the current approximant of order $(k-1, q-1)$. At each iteration we locate the sample (s_i, p_j) that gives the largest error $|H(s_i, p_j) - \tilde{H}(s_i, p_j)|$. Denote this tuple by $(\sigma_{k+1}, \pi_{q+1})$. If the frequency s_i has not been chosen before, it is added to the set $\{\sigma_i\}$ (so $k \leftarrow k+1$); if the parameter p_j has not been chosen before, it is added to $\{\pi_j\}$ (so $q \leftarrow q+1$). This allows the two orders to grow independently, automatically adapting to the complexity of H in each variable. Once the partition is updated, new coefficients β_{ij} are set via (3.4) and the least-squares problem (3.10) is solved for the updated vector $\mathbf{a}$. The process is repeated until a desired tolerance or maximum orders are attained. A sketch of the algorithm is given in Algorithm 3.1.

4.3 Relation to AAA

The parallelism between the two algorithms is evident:

- Both use a barycentric representation that guarantees interpolation at the chosen support points and a linearised least-squares fit for the remaining data.

- Both rely on a Loewner matrix (standard in AAA, block-structured in p-AAA) to formulate the linearised problem.
- Both employ a greedy selection of new interpolation points; p-AAA's selection is a natural extension to two dimensions, updating each variable independently.

In the special case where the parameter space consists of a single point ($M = 1$) the two-variable barycentric form (3.2) collapses to the single-variable form (2.2), the matrix L_2 reduces to the standard Loewner matrix L, and p-AAA becomes exactly AAA. Hence p-AAA is a genuine multivariate extension of AAA, inheriting its adaptivity, numerical robustness, and ability to blend interpolation with least-squares fitting.

As noted in the literature, if the orders k and q are chosen sufficiently large, the matrix L_2 may have a nontrivial null space; then $\mathbf{a}$ can be taken from that null space, leading to an interpolant of the full data set – this is the parametric Loewner framework. p-AAA, on the other hand, builds the approximant iteratively by greedily selecting a subset of the data for interpolation and fitting the rest in a least-squares sense. This adaptive data partitioning is a key feature inherited from AAA and distinguishes p-AAA from the one-step Loewner approach.

5 The AAA^2 as an Extension of AAA

In signal and power integrity analysis, constructing rational transfer functions from frequency-domain responses (either from simulations or measurements) is a crucial step for time-domain simulation. The AAA (Adaptive Antoulas–Anderson) algorithm has emerged as an attractive tool for rational approximation due to its numerical stability, absence of initial pole guesses, and adaptive order selection. However, the standard AAA algorithm **neither guarantees real coefficients nor ensures that all poles lie in the left half-plane (i.e., stability)**—properties that are indispensable for modeling passive electrical networks. To address these limitations, Valera-Rivera and Engin introduced the AAA^2 algorithm [4], an extension of AAA that generates rational transfer functions with **real coefficients and stable poles**.

5.1 Recollection of the AAA Algorithm

The AAA algorithm represents the rational approximant in barycentric form:

$$r(s) = \frac{n(s)}{d(s)} = \frac{\sum_{i=0}^{N} \dfrac{a_i H_i}{s - j\omega_i}}{\sum_{i=0}^{N} \dfrac{a_i}{s - j\omega_i}}, \tag{1}$$

where $H_i = H(j\omega_i)$ are the given function values at the sample frequencies, and a_i are unknown weights. By greedily selecting nodes $j\omega_i$ (choosing the frequency where the current approximation error is largest) and solving a linearized least-squares problem involving a Loewner matrix, AAA obtains high-accuracy approximations without requiring an initial pole guess. Nevertheless, the coefficients in (1) are generally complex, and poles may appear in the right half-plane, which limits the applicability of AAA in passive circuit modeling.

5.2 Enforcing Conjugate Symmetry and Real Coefficients

To obtain a transfer function with real coefficients, AAA^2 explicitly incorporates complex-conjugate terms into the barycentric form:

$$r(s) = \frac{\sum_{i=0}^{N} \dfrac{a_i H_i}{s - j\omega_i} + \sum_{i=0}^{N} \dfrac{a_i^* H_i^*}{s + j\omega_i}}{\sum_{i=0}^{N} \dfrac{a_i}{s - j\omega_i} + \sum_{i=0}^{N} \dfrac{a_i^*}{s + j\omega_i}}. \tag{4}$$

This representation guarantees that $r(s)$ takes real values for real or complex-conjugate arguments, thereby automatically satisfying the real-coefficient requirement. The weights a_i may be complex,

but they can also be restricted to be real or purely imaginary, leading to different approximation characteristics.

In particular, choosing purely imaginary weights $a_i = ja_i''$ transforms (4) into a form expressed in terms of the squared variable s^2:

$$r(s) = \frac{\sum_{i=0}^{N} \frac{a_i \omega_i H_i'}{s^2 + \omega_i^2} + s \sum_{i=0}^{N} \frac{a_i H_i''}{s^2 + \omega_i^2}}{\sum_{i=0}^{N} \frac{a_i \omega_i}{s^2 + \omega_i^2}}, \tag{5}$$

where $H_i = H_i' + jH_i''$. Because the fitting is now performed in the s^2-domain, this variant is named the AAA^2 algorithm. An interesting feature of (5) is that it naturally separates the rational function into an even part $r_e(s)$ and an odd part $r_o(s)$, which correspond respectively to the real and imaginary parts of the frequency response.

5.3 Modified Loewner Matrix and Least-Squares Solution

With the introduction of conjugate nodes, the Loewner matrix must be modified accordingly. Define the original Cauchy matrix $C_{ki} = 1/(j\hat{\omega}_k - j\omega_i)$ and the conjugate Cauchy matrix $C_{ki}^{+} = 1/(j\hat{\omega}_k + j\omega_i)$. The two associated Loewner matrices are

$$A_1 = \hat{H}C - CH, \quad A_2 = \hat{H}C^{+} - C^{+}H^{*}, \tag{6}$$

where $\hat{H} = \mathrm{diag}(\hat{H}_0, \ldots, \hat{H}_M)$ and $H = \mathrm{diag}(H_0, \ldots, H_N)$. The least-squares problem now becomes

$$\min \|A_1 a + A_2 a^*\| = \min \|(A_1 + A_2)a' + j(A_1 - A_2)a''\|, \quad \|[a'; a'']\| = 1. \tag{7}$$

By stacking the real and imaginary parts, we obtain a modified real Loewner matrix:

$$\mathcal{A} = \begin{pmatrix} A' \\ A'' \end{pmatrix}, \quad \text{where } A = A' + jA'', \tag{8}$$

and solve

$$\min \left\| \mathcal{A} \begin{pmatrix} a' \\ a'' \end{pmatrix} \right\|, \quad \left\| \begin{pmatrix} a' \\ a'' \end{pmatrix} \right\| = 1,$$

yielding real vectors a', a'' that respect the required conjugate symmetry.

For multiport networks (matrix-valued functions), the modified Loewner matrices of each port can be stacked vertically, and the same weight vector is solved for, producing a transfer function matrix with **common poles**.

5.4 Pole Extraction and Stability Enforcement

The poles of the AAA^2 approximant are obtained by solving a generalized eigenvalue problem. For the general form (4), the poles are the eigenvalues of the matrix $\delta - c^T b$ (after discarding the zero eigenvalue). For numerical stability, frequency scaling is applied.

In the case of purely imaginary weights, the denominator factorizes as

$$\sum_{i=0}^{N} \frac{a_i \omega_i}{s^2 + \omega_i^2} = a \frac{\prod_{i=0}^{N-1}(s^2 - p_i^2)}{\prod_{i=0}^{N}(s^2 + \omega_i^2)}, \tag{13}$$

revealing that poles appear in pairs $\pm p_i$. The unstable poles (with positive real part) actually belong to $r(-s)$ and can be safely discarded. If purely imaginary poles (on the imaginary axis) occur, they are handled according to the frequency range: poles inside the data band are rejected, while those outside may be perturbed by adding a small negative real part to ensure stability.

Once stable poles are obtained, residues are extracted via a least-squares fit similar to vector fitting, yielding the final partial-fraction representation:

$$r(s) = e + ds + \sum_{i=0}^{N-1} \frac{k_i}{s - p_i}, \tag{15}$$

which is convenient for subsequent passivity assessment and time-domain integration.

5.5 Relation to AAA

The AAA^2 algorithm fully inherits the core framework of AAA: it still employs a barycentric representation, greedily selects nodes, and solves a linearized least-squares problem via an SVD of a Loewner matrix. The only difference is that AAA^2 explicitly incorporates conjugate symmetry when constructing the Loewner matrix, thereby enforcing real coefficients, and it subsequently ensures stability by appropriate pole selection and discarding mechanisms. Thus, AAA^2 can be regarded as a natural and necessary extension of AAA in the context of circuit and electromagnetic modeling—it preserves all the advantages of AAA (numerical stability, no initial guess, adaptive order) while remedying its deficiencies in real-coefficient and stability guarantees.

Numerical experiments demonstrate that for the noise-free ISS 1R module data, AAA^2 (with purely imaginary weights) achieves accuracy comparable to standard AAA (RMS error 8.8×10^{-7}) at the same order, while producing only stable poles; for measured common-mode filter data, AAA^2 (with complex weights) attains a fitting accuracy similar to vector fitting (RMS error 0.0013). These results validate AAA^2 as an effective and practical extension of AAA.

6 Conclusions

In this paper, we have presented a comprehensive overview of the AAA algorithm and three of its most important extensions: continuum AAA, p-AAA, and AAA^2. Each of these extensions addresses specific limitations of the original algorithm while preserving its core advantages of numerical stability, adaptive order selection, and the ability to blend interpolation with least-squares fitting.

The original AAA algorithm, with its barycentric representation and greedy selection strategy, has proven to be remarkably effective for rational approximation on discrete point sets. Its numerical stability stems from the barycentric form, which avoids the catastrophic cancellation that plagues polynomial representations when zeros and poles cluster near singularities. The algorithm's adaptive nature, automatically determining a suitable degree by monitoring the error, makes it accessible to users who need only supply a tolerance.

Continuum AAA represents a natural evolution of the algorithm, addressing the philosophical and practical limitations of requiring a user-supplied discrete grid. By adaptively generating sample points at each iteration based on the current support point distribution, the algorithm can work directly on continuous domains without manual discretization. The automatic refinement near regions of large error leads to exponential clustering of points near singularities—a key property for high-accuracy rational approximation. The detection and management of spurious poles, with fallback to the last successful pole-free approximation, ensures that the returned rational function is analytic on the desired domain, a crucial feature for applications in control theory and model order reduction.

The p-AAA algorithm extends AAA to multivariate settings, addressing the growing need for data-driven modeling of parametric dynamical systems. By introducing a two-variable barycentric form and constructing a block-structured two-dimensional Loewner matrix, p-AAA inherits all the advantages of the original algorithm while enabling approximation of functions that depend on both frequency and physical parameters. The independent greedy selection in each variable allows the orders to grow according to the complexity of the function in each dimension, providing remarkable flexibility. The connection to the parametric Loewner framework further illuminates the relationship between interpolation-based and least-squares-based approaches.

The AAA^2 addresses the specific requirements of circuit and electromagnetic modeling, where real coefficients and stable poles are indispensable. By explicitly incorporating conjugate symmetry into the barycentric representation, the algorithm guarantees real coefficients. The careful handling of poles—discarding unstable poles and appropriately treating purely imaginary poles—ensures that the resulting rational function is suitable for time-domain simulation of passive electrical networks.

Numerical experiments confirm that AAA^2 achieves accuracy comparable to standard AAA while satisfying the additional constraints required for practical applications.

Together, these extensions demonstrate the remarkable adaptability of the AAA framework. Each extension preserves the core algorithmic structure—barycentric representation, greedy selection, and linearized least-squares solution via SVD—while modifying specific components to address particular application domains. This modularity suggests that the AAA framework may continue to evolve to meet emerging challenges in scientific computing.

Looking forward, several directions for future research are apparent. For continuum AAA, further improvements in handling singularities and reducing floating-point limitations could enhance reliability for extremely challenging problems. For p-AAA, extensions to functions of more than two variables and connections to other multivariate approximation methods warrant investigation. For AAA^2, integration with passivity enforcement techniques and extension to a broader class of network problems could increase its utility. More broadly, the success of these extensions raises the question of whether similar adaptive, barycentric-based approaches could be developed for other approximation problems, such as fitting by sums of exponentials or Gaussians.

The AAA algorithm and its extensions have fundamentally changed the landscape of rational approximation, transforming what was once considered a difficult problem into a routine computation. As these methods continue to mature and find new applications, they promise to remain indispensable tools in numerical analysis and scientific computing for years to come.

References

[1] Nakatsukasa, Yuji, et al. "The AAA algorithm for rational approximation." in SIAM Journal on Scientific Computing, vol. 40, no. 3, 2018, pp. A1494-A1522.

[2] Carracedo Rodríguez, Andrea, et al. "The p-AAA algorithm for data driven modeling of parametric dynamical systems." arXiv preprint arXiv:2003.06536 (2022).

[3] Driscoll, Toby, Yuji Nakatsukasa, and Lloyd N. Trefethen. "AAA rational approximation on a continuum." arXiv preprint arXiv:2305.03677 (2023).

[4] A. Valera-Rivera and A. E. Engin, "AAA Algorithm for Rational Transfer Function Approximation With Stable Poles," in IEEE Letters on Electromagnetic Compatibility Practice and Applications, vol. 3, no. 3, pp. 92-95, Sept. 2021